Análisis del Lenguaje de Programación

Licenciatura en Ciencias de la Computación

Lenguaje de Dominio Específico de Distribuciones de Probabilidad y Cadenas de Markov

Trabajo Práctico Final

Eliseo Feuli

Marzo 2026

Índice

1 Introducción **3**

2 Resumen de Funciones y Distribuciones **3**
2.1 Funciones Estadísticas . . . 3
2.2 Variables Aleatorias Discretas . . . 3
2.3 Variables Aleatorias Continuas . . . 4

3 Cadenas de Markov **4**
3.1 Definición . . . 4
3.2 Cálculo de Probabilidades . . . 4
3.3 Análisis a Largo Plazo y Convergencia . . . 4
3.3.1 Probabilidades de Alcance (`F(X, i, j)`) . . . 5
3.3.2 Distribución Estacionaria (`stationary`) . . . 5

4 Sintaxis **5**
4.1 Programa y Comandos . . . 5
4.2 Declaraciones . . . 6
4.3 Expresiones Generales . . . 6
4.4 Expresiones Numéricas . . . 6
4.5 Estadísticas . . . 6
4.6 Expresiones Vectoriales . . . 6
4.7 Variables Aleatorias . . . 7
4.8 Probabilidades . . . 7
4.9 Cadenas de Markov . . . 7
4.10 Caminos . . . 7

5 Parser **7**

6 Semántica: Representación en Haskell **8**
6.1 Variables Aleatorias (valores) . . . 8
6.2 Cadenas de Markov (valores) . . . 8
6.3 Operadores . . . 8
6.4 Expresiones de Variables Aleatorias . . . 9
6.5 Distribuciones Discretas y Continuas . . . 9
6.6 Expresiones . . . 9
6.7 Valores . . . 10
6.8 Tipos . . . 10
6.9 Comandos . . . 10

7 Verificador de Tipos (TypeChecker) **10**
7.1 Mecanismo de Inferencia y Estado Global . . . 10
7.2 Reglas de Tipado . . . 11
7.3 Errores Detectados . . . 11

8 Semántica Operacional **11**
8.1 Literales y Variables . . . 11
8.2 Evaluación de Variables Aleatorias . . . 12
8.2.1 Distribuciones Discretas . . . 12
8.2.2 Distribuciones Continuas . . . 12
8.3 Operaciones Numéricas . . . 12
8.4 Acceso a Vectores . . . 12
8.5 Estadísticas . . . 13
8.6 Probabilidades . . . 13

8.7 Cadenas de Markov . . . 13
8.8 Simulaciones . . . 13
8.9 Comandos . . . 14

9 Validador **14**
9.1 Valores Numéricos . . . 14
9.2 Variables Aleatorias Discretas . . . 14
9.2.1 Binomial Bin(n, p) . . . 14
9.2.2 Poisson Poi(λ) . . . 14
9.2.3 Geométrica Geo(p) . . . 14
9.2.4 Pascal BN(r, p) . . . 14
9.2.5 Hipergeométrica HG(m, r, n) . . . 14
9.2.6 Distribución Personalizada [xs ps] . . . 15
9.3 Variables Aleatorias Continuas . . . 15
9.3.1 Normal N(μ, σ) . . . 15
9.3.2 Uniforme Unif(a, b) . . . 15
9.3.3 Exponencial Exp(λ) . . . 15
9.4 Probabilidades . . . 15
9.5 Cadenas de Markov . . . 15
9.5.1 Definición de la cadena . . . 15
9.5.2 Vector de distribución inicial . . . 15
9.5.3 Nombres de estado . . . 15
9.5.4 Caminos . . . 16
9.5.5 Número de pasos . . . 16

10 Pretty Printer **16**

11 Visualización y Representación de Datos **16**
11.1 Generación de Tablas . . . 16
11.2 Generación de Gráficos . . . 16

12 Arquitectura de la Mónada: `MonadProb` **16**

13 Estructura del Proyecto **18**
13.1 Organización de Archivos . . . 18

14 Ejecución **18**
14.1 Paso de Archivos y Configuración . . . 18
14.2 Ejemplos . . . 19
14.3 Modo Interactivo (REPL) . . . 19
14.4 Ejemplo de Sesión . . . 19

15 Bibliografía y Recursos **20**

16 Mejoras futuras **20**

1 Introducción

Este trabajo presenta el diseño e implementación de un lenguaje de dominio específico (DSL) para el cálculo y análisis de distribuciones de probabilidad y cadenas de Markov. El lenguaje permite declarar variables aleatorias discretas y continuas, calcular estadísticas (esperanza, varianza, moda, entre otras), consultar probabilidades con distintos operadores de comparación, y modelar procesos estocásticos mediante cadenas de Markov.

El intérprete está implementado en Haskell y sigue una arquitectura de: **parsing**, **verificación de tipos**, **validación matemática** y **evaluación**. El estado del intérprete se gestiona mediante una mónada personalizada `ProbM` que combina lectura de configuración, estado mutable, manejo de errores y efectos de E/S.

A continuación se describe la sintaxis del lenguaje, las distribuciones soportadas, el mecanismo de tipos, la validación y la arquitectura interna.

2 Resumen de Funciones y Distribuciones

2.1 Funciones Estadísticas

A continuación se describen las funciones de cálculo estadístico implementadas para el tipo `RandExp`.

Función	Descripción	Fórmula
`E`	Esperanza matemática o valor promedio.	$E[X] = \sum x\, P(x)$
`V`	Medida de la dispersión de la variable.	$\mathrm{Var}(X) = E[X^2] - (E[X])^2$
`SD`	Desviación estándar (raíz de la varianza).	$\sigma = \sqrt{\mathrm{Var}(X)}$
`mode`	Valor con la mayor probabilidad de ocurrencia.	$\arg\max_x f(x)$
`pdf`	Función de densidad (continua).	$f(x)$
`maxP`	Valor máximo de la probabilidad (discretas).	$\max P(X{=}x)$
`maxPDF`	Valor máximo de la densidad (continuas).	$\max f(x)$

Table 1: Funciones estadísticas disponibles.

2.2 Variables Aleatorias Discretas

Representan procesos donde los resultados pertenecen a un conjunto numerable.

- **Binomial** `Bin(n, p)`**:** Modela el número de éxitos en n ensayos independientes con probabilidad p.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

- **Poisson** `Poi(λ)`**:** Expresa la probabilidad de que ocurra un número determinado de eventos en un intervalo fijo.

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

- **Geométrica** `Geo(p)`**:** Número de ensayos necesarios hasta obtener el primer éxito.

$$P(X = k) = (1-p)^{k-1} p$$

- **Pascal** `BN(r, p)`**:** También conocida como Binomial Negativa; número de ensayos necesarios para obtener r éxitos.

$$P(X = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}$$

- **Hipergeométrica** `HG(m, r, n)`**:** Probabilidad de obtener k éxitos en una muestra de tamaño n extraída sin reposición de una población de m elementos, de los cuales r tienen la característica deseada.

$$P(X = k) = \frac{\binom{r}{k}\binom{m-r}{n-k}}{\binom{m}{n}}$$

- **Personalizada** `[xs ps]`**:** Distribución definida por el usuario mediante un vector de valores y un vector de probabilidades asociadas.

2.3 Variables Aleatorias Continuas

Variables que pueden tomar cualquier valor dentro de un intervalo real.

- **Normal** `N(`μ`, `σ`)`**:** Distribución simétrica definida por su media μ y desviación estándar $\sigma > 0$.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

- **Uniforme** `Unif(a, b)`**:** Todos los valores en $[a, b]$ tienen la misma densidad.

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

- **Exponencial** `Exp(`λ`)`**:** Describe el tiempo entre eventos en un proceso de Poisson.

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

3 Cadenas de Markov

3.1 Definición

Una cadena de Markov es un modelo estocástico que cumple la propiedad de Markov (falta de memoria):

$$P(X_{n+1} = x \mid X_n, X_{n-1}, \ldots, X_0) = P(X_{n+1} = x \mid X_n)$$

3.2 Cálculo de Probabilidades

- **Probabilidad de una trayectoria:** Para un camino $x_0 \to x_1 \to \cdots \to x_k$:

$$P(\text{path}) = P(X_0 = x_0) \prod_{t=1}^{k} P(X_t = x_t \mid X_{t-1} = x_{t-1})$$

- **Transición en n pasos:** La probabilidad de pasar del estado i al j en n pasos está dada por la entrada (i, j) de la matriz P^n.

3.3 Análisis a Largo Plazo y Convergencia

El núcleo de la implementación transforma propiedades estocásticas en sistemas lineales $A\mathbf{x} = \mathbf{b}$, resueltos mediante **eliminación de Gauss-Jordan**.

3.3.1 Probabilidades de Alcance (`F(X, i, j)`)

La probabilidad de alcance h_{ij} es la probabilidad de que el proceso llegue eventualmente al estado j partiendo desde i. Se define recursivamente:

1. **Caso base:** $h_{jj} = 1$
2. **Caso recursivo:** $h_{ij} = \sum_k p_{ik}\, h_{kj}$

Esto da lugar al sistema $(I - P)H_j = \mathbf{0}$. La fila j se reemplaza por la fila correspondiente de la identidad, garantizando que la solución satisfaga $h_{jj} = 1$.

3.3.2 Distribución Estacionaria (`stationary`)

La distribución estacionaria π es el vector de probabilidad tal que $\pi P = \pi$, equivalente a:

$$\pi(P - I) = \mathbf{0}$$

Como las columnas de $(P - I)$ son linealmente dependientes (cada fila de P suma 1), el sistema tiene infinitas soluciones. Para obtener la única distribución de probabilidad válida se impone la condición de normalización $\sum_i \pi_i = 1$.

En la implementación se trabaja con la transpuesta $(P^T - I)$ para operar con vectores columna. La última fila de la matriz se reemplaza por un vector de unos y el último elemento del vector de resultados por 1, forzando a Gauss-Jordan a encontrar la solución normalizada.

4 Sintaxis

La sintaxis permite definir y operar sobre números, vectores, variables aleatorias y cadenas de Markov. Cada comando debe ocupar exactamente una línea; el parser identifica el fin de una instrucción al encontrar el carácter de nueva línea.

4.1 Programa y Comandos

⟨*program*⟩ ::= { *command* }

⟨*command*⟩ ::= ⟨*declaration*⟩
| ‘`print`’ ‘`(`’ *expr* ‘`)`’
| ‘`table`’ ‘`(`’ *randExpr* ‘`)`’
| ‘`table`’ ‘`(`’ *randExpr* ‘`,`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’
| ‘`plot`’ ‘`(`’ *randExpr* ‘`)`’
| ‘`plot`’ ‘`(`’ *markovExpr* ‘`)`’

Comando	**Descripción**
`x = e`	Liga el nombre `x` al valor de `e`
`X ~ e`	Declara la variable aleatoria `X`
`x -> n`	Liga el nombre `x` a un nodo de Markov
`print(e)`	Evalúa e imprime `e`
`table(e)`	Tabla de distribución de `e`
`table(e, lo, hi)`	Tabla de distribución en el rango $[lo, hi]$
`plot(e)`	Grafica la distribución o cadena `e`

Table 2: Comandos del lenguaje.

4.2 Declaraciones

⟨*declaration*⟩ ::= *lowerName* '`=`' *numExpr*
| *lowerName* '`=`' *vecExpr*
| *lowerName* '`=`' *pathExpr*
| *upperName* '`~`' *randExpr*
| *upperName* '`:=`' *randExpr*
| *upperName* '`=`' *markovExpr*
| *name* '`->`' *nodeExpr*

Los identificadores en minúscula (*lowerName*) se usan para variables numéricas, vectoriales y de camino. Los identificadores en mayúscula (*upperName*) se reservan para variables aleatorias y cadenas de Markov.

4.3 Expresiones Generales

⟨*expr*⟩ ::= *numExpr*
| *vecExpr*
| *randExpr*
| *markovExpr*
| *pathExpr*

4.4 Expresiones Numéricas

⟨*numExpr*⟩ ::= *numTerm* '`+`' *numTerm*
| *numTerm* '`-`' *numTerm*

⟨*numTerm*⟩ ::= *numAtom* '`*`' *numAtom*
| *numAtom* '`/`' *numAtom*

⟨*numAtom*⟩ ::= '`-`' *numBaseAtom*
| *numBaseAtom*

⟨*numBaseAtom*⟩ ::= *number*
| *lowerName*
| *probExpr*
| *vecExpr* '`[`' *numExpr* '`]`'
| *numStat*
| '`(`' *numExpr* '`)`'

4.5 Estadísticas

⟨*numStat*⟩ ::= '`E`' '`(`' *randExpr* '`)`'
| '`V`' '`(`' *randExpr* '`)`'
| '`SD`' '`(`' *randExpr* '`)`'
| '`pdf`' '`(`' *randExpr* '`,`' *numExpr* '`)`'
| '`maxP`' '`(`' *randExpr* '`)`'
| '`maxPDF`' '`(`' *randExpr* '`)`'

⟨*vecStat*⟩ ::= '`mode`' '`(`' *randExpr* '`)`'

4.6 Expresiones Vectoriales

⟨*vecExpr*⟩ ::= '`(`' *numExpr* { '`,`' *numExpr* } '`)`'
| *lowerName*
| *vecStat*

4.7 Variables Aleatorias

⟨*discRand*⟩ ::= (‘`Bin`’ | ‘`Binomial`’) ‘`(`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’
| (‘`Poi`’ | ‘`Poisson`’) ‘`(`’ *numExpr* ‘`)`’
| (‘`Geo`’ | ‘`Geometric`’) ‘`(`’ *numExpr* ‘`)`’
| (‘`BN`’ | ‘`Pascal`’) ‘`(`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’
| (‘`HG`’ | ‘`Hipergeometric`’) ‘`(`’ *numExpr* ‘`,`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’
| ‘`[`’ *vecExpr* ‘`,`’ *vecExpr* ‘`]`’

⟨*contRand*⟩ ::= (‘`N`’ | ‘`Normal`’) ‘`(`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’
| (‘`Exp`’ | ‘`Exponential`’) ‘`(`’ *numExpr* ‘`)`’
| (‘`Unif`’| ‘`Uniform`’) ‘`(`’ *numExpr* ‘`,`’ *numExpr* ‘`)`’

⟨*randExpr*⟩ ::= *upperName*
| *discRand*
| *contRand*

4.8 Probabilidades

⟨*probExpr*⟩ ::= ‘`P`’ ‘`(`’ *probRandExpr* ‘`)`’
| ‘`F`’ *probMarkovExpr*

⟨*probRandExpr*⟩ ::= *numExpr opComp randExpr opComp numExpr*
| *numExpr opComp randExpr*
| *randExpr opComp numExpr*

⟨*probMarkovExpr*⟩ ::= *numExpr* ‘`(`’ *markovExpr* ‘`,`’ *name* ‘`,`’ *name* ‘`)`’
| ‘`(`’ *markovExpr* ‘`,`’ *pathExpr* ‘`)`’
| ‘`(`’ *markovExpr* ‘`,`’ *name* ‘`,`’ *name* ‘`)`’

⟨*opComp*⟩ ::= ‘`<=`’ | ‘`<`’ | ‘`>=`’ | ‘`>`’ | ‘`/=`’ | ‘`=`’

4.9 Cadenas de Markov

⟨*markovExpr*⟩ ::= ‘`mk`’ ‘`(`’ *name* { ‘`,`’ *name* } ‘`)`’
| *upperName*
| ‘`ex`’ ‘`(`’ *markovExpr* ‘`,`’ *numExpr* ‘`)`’
| ‘`stationary`’ ‘`(`’ *markovExpr* ‘`)`’

⟨*nodeExpr*⟩ ::= ‘`[`’ *neigh* { ‘`,`’ *neigh* } ‘`]`’

⟨*neigh*⟩ ::= ‘`(`’ *name* ‘`,`’ *numExpr* ‘`)`’

4.10 Caminos

⟨*pathExpr*⟩ ::= ‘`[`’ *name* { ‘`,`’ *name* } ‘`]`’
| *lowerName*
| ‘`simulate`’ ‘`(`’ *markovExpr* ‘`,`’ *numExpr* ‘`,`’ ‘`start`’ ‘`=`’ *name* ‘`)`’
| ‘`simulate`’ ‘`(`’ *markovExpr* ‘`,`’ *numExpr* ‘`,`’ ‘`prob`’ ‘`=`’ *vecExpr* ‘`)`’

5 Parser

El parser está implementado con la librería `Parsec`, utilizando el módulo `Text.Parsec.Token` para la definición del lexer. Los aspectos más relevantes del diseño son:

- **Un comando por línea.** El parser exige que cada instrucción esté separada por al menos un salto de línea.
- **Comentarios.** Se soportan comentarios de línea (`//`) y de bloque (`/* ... */`), ignorados por el lexer.

- **Distinción mayúscula/minúscula.** Los identificadores en mayúscula (*upperName*) se reservan para variables aleatorias y cadenas de Markov; los identificadores en minúscula (*lowerName*) para variables numéricas, vectoriales y de camino.
- **Precedencia aritmética.** Las expresiones numéricas se parsean con la jerarquía estándar: multiplicación y división tienen mayor precedencia que suma y resta, implementado mediante `chainl1` con dos niveles de gramática (*numTerm* y *numExpr*).

6 Semántica: Representación en Haskell

Para representar la semántica del lenguaje en Haskell se definen los siguientes tipos de datos.

6.1 Variables Aleatorias (valores)

```
data VarDisc
  = Bin    Int    Double            -- Binomial(n, p)
  | Poiss  Double                   -- Poisson(lambda)
  | Geo    Double                   -- Geometric(p)
  | Pasc   Int    Double            -- Pascal(r, p)
  | Hiper  Int    Int    Int        -- HiperGeometric(m, r, n)
  | Custom (Vec Int) (Vec Double)   -- Custom(values, probs)
  deriving (Show, Eq)

data VarCont
  = Norm Double Double   -- Normal(mu, sigma)
  | Unif Double Double   -- Uniform(a, b)
  | Expo Double          -- Exponential(lambda)
  deriving (Show, Eq)

data RandVar
  = Cont VarCont
  | Disc VarDisc
  deriving (Show, Eq)
```

6.2 Cadenas de Markov (valores)

```
data Markov = Mk (Vec Name) (Matrix Double)
  deriving (Show, Eq)

newtype NodeVal = N [(Name, Double)]
  deriving (Show, Eq)
```

6.3 Operadores

```
data OpComp = Lt | Gt | Lte | Gte | Eq | NEq
  deriving (Show, Eq)

data OpBin = Plus | Minus | Times | Div
  deriving (Show, Eq)
```

6.4 Expresiones de Variables Aleatorias

Antes de la evaluación, las expresiones de variables aleatorias se representan como:

```
data RandExp  = ContE ExpCont | DiscE ExpDisc  deriving (Show, Eq)
newtype NodeExp   = NE [(Name, Exp)]           deriving (Show, Eq)
newtype MarkovExp = MarkovE Path               deriving (Show, Eq)
```

6.5 Distribuciones Discretas y Continuas

```
data ExpDisc
  = BinE    Exp Exp
  | PoissE  Exp
  | GeoE    Exp
  | PascE   Exp Exp
  | HiperE  Exp Exp Exp
  | CustomE Exp Exp
  deriving (Show, Eq)

data ExpCont
  = NormE Exp Exp
  | UnifE Exp Exp
  | ExpoE Exp
  deriving (Show, Eq)
```

6.6 Expresiones

Este es el tipo principal con el que trabaja el intérprete.

```
data Exp
  -- Variables y literales
  = VarRef Name           -- Double | Vec Double | RandVar | Markov | Path
  | ConstN Double         -- constante numerica
  | UMinus Exp            -- negacion unaria
  | OpNum  OpBin Exp Exp  -- operacion binaria numerica
  -- Estadisticas
  | Mean     Exp          -- E(X)
  | Variance Exp          -- V(X)
  | StdDev   Exp          -- SD(X)
  | Mode     Exp          -- mode(X)     :: Vec Double
  | PDF      Exp Exp      -- pdf(X, x)   :: Double
  | MaxP     Exp          -- maxP(X)     :: Double  (discreta)
  | MaxPDF   Exp          -- maxPDF(X)   :: Double  (continua)
  -- Probabilidades
  | Prob        Exp OpComp Exp              -- P(X op n)
  | ProbBetween Exp OpComp Exp OpComp Exp   -- P(n1 op X op n2)
  -- Vectores
  | ConstV (Vec Exp)      -- literal vectorial
  | Access Exp Exp        -- v[i]
  -- Aleatorias
  | Rand RandExp
  -- Cadenas de Markov
  | ConstCh  Path
  | Markov   MarkovExp
  | ProbStep Exp Name Name Exp   -- F^n(X, i, j)
```

```
  | ProbPath Exp Exp             -- F(X, path)
  | ProbHit  Exp Name Name       -- F(X, i, j)  hitting
  | NextDist    Exp Exp          -- ex(X, n)
  | Stationary  Exp              -- stationary(X)
  | SimulFromName Exp Name Exp   -- simulate(X, n, start = s)
  | SimulFromVec  Exp Exp  Exp   -- simulate(X, n, prob  = v)
  deriving (Show, Eq)
```

6.7 Valores

Los valores son el resultado de reducir una expresión.

```
data Value
  = VNum  Double
  | VVec  (Vec Double)
  | VRand RandVar
  | VMark Markov
  | VNode NodeVal
  | VPath Path
  | Dummy           -- valor centinela usado durante el typechecking
  deriving (Show, Eq)
```

6.8 Tipos

```
data Type
  = Num | Vec | RandDisc | RandCont | Markov | Path | Node
  deriving (Show, Eq)
```

6.9 Comandos

```
data Comm
  = Print  Exp
  | Let    Name Exp
  | LetN   Name NodeExp
  | Table  Exp
  | TableR Exp Exp Exp
  | Plot   Exp
  deriving (Show, Eq)
```

7 Verificador de Tipos (TypeChecker)

El módulo de typechecking valida que los comandos y expresiones parseados cumplan con las reglas de tipado antes de su ejecución, previniendo fallos en tiempo de evaluación.

7.1 Mecanismo de Inferencia y Estado Global

Cuando el programa procesa una declaración, el verificador realiza las siguientes acciones:

- **Inferencia de tipos:** El tipo de una variable no se declara explícitamente; el sistema lo deduce analizando la estructura de la expresión asignada.

- **Registro en el contexto:** Una vez inferido el tipo, la variable se registra en el estado global con su tipo y un valor `Dummy` como marcador. Esto permite detectar usos de variables no inicializadas y referencias hacia adelante dentro del mismo programa.

7.2 Reglas de Tipado

La función principal `checkType :: Exp -> m Type` infiere el tipo de cada expresión según la siguiente tabla:

Expresión	**Tipo inferido**
`ConstN`, `UMinus`, `OpNum`	`Num`
`Mean`, `Variance`, `StdDev`	`Num`
`PDF`, `MaxP`, `MaxPDF`	`Num`
`Prob`, `ProbBetween`	`Num`
`ProbStep`, `ProbPath`, `ProbHit`	`Num`
`Mode`, `Stationary`, `ConstV`	`Vec`
`Rand (DiscE _)`	`RandDisc`
`Rand (ContE _)`	`RandCont`
`Markov`, `NextDist`	`Markov`
`ConstCh`, `SimulFromName`, `SimulFromVec`	`Path`

Table 3: Reglas de inferencia de tipos.

7.3 Errores Detectados

- **Variables no definidas:** Toda variable presente en una expresión debe haber sido declarada previamente en el contexto.
- **Inconsistencia de tipos:** Se detectan operaciones inválidas como pasar una variable discreta donde se espera una continua, o usar un escalar donde se requiere un vector.
- **Caminos vacíos:** Un literal de camino debe contener al menos un estado.
- **Operadores inválidos en `ProbBetween`:** Solo se aceptan combinaciones del tipo k_1 $(\geq \mid >)$ X $(\leq \mid <)$ k_2.

8 Semántica Operacional

Las siguientes reglas definen la semántica de paso grande (*big-step*) del lenguaje. Se usa la notación $\langle e, \sigma \rangle \Downarrow v$ para indicar que la expresión e evalúa al valor v bajo el entorno σ.

Se usa $\lfloor n \rfloor$ para denotar la conversión de `Double` a `Int`.
El símbolo $p \sim \text{simulate}(M, \cdot, k)$ indica que el camino p se obtiene de forma aleatoria siguiendo las probabilidades de transición de M durante k pasos, partiendo del estado inicial dado por nombre o por la distribución inicial π_0, respectivamente.
Tener en cuenta que a las variables aleatorias y a las probabilidades se le aplican validaciones para controlar que son expresiones bien formadas.

8.1 Literales y Variables

$$\frac{}{\langle \texttt{ConstN}\ n,\ \sigma \rangle \Downarrow \texttt{VNum}\ n} \qquad \frac{\sigma_{decls}(x) = v}{\langle \texttt{VarRef}\ x,\ \sigma \rangle \Downarrow v}$$

$$\frac{}{\langle \texttt{ConstCh}\ p,\ \sigma \rangle \Downarrow \texttt{VPath}\ p} \qquad \frac{}{\langle \texttt{ConstV}\ v,\ \sigma \rangle \Downarrow \texttt{VVec}\ v}$$

8.2 Evaluación de Variables Aleatorias

$$\frac{}{\langle \texttt{RandE}\ v,\ \sigma\rangle \Downarrow \texttt{RandExp}\ v}$$

8.2.1 Distribuciones Discretas

$$\frac{\langle e_1,\sigma\rangle \Downarrow n_1 \quad \langle e_2,\sigma\rangle \Downarrow n_2}{\langle \texttt{BinE}\ e_1\ e_2,\sigma\rangle \Downarrow \texttt{Bin}\ \lfloor n_1\rfloor\ n_2}$$

$$\frac{\langle e,\sigma\rangle \Downarrow n}{\langle \texttt{PoissE}\ e,\sigma\rangle \Downarrow \texttt{Poiss}\ n} \qquad \frac{\langle e,\sigma\rangle \Downarrow n}{\langle \texttt{GeoE}\ e,\sigma\rangle \Downarrow \texttt{Geo}\ n}$$

$$\frac{\langle e_1,\sigma\rangle \Downarrow n_1 \quad \langle e_2,\sigma\rangle \Downarrow n_2}{\langle \texttt{PascE}\ e_1\ e_2,\sigma\rangle \Downarrow \texttt{Pasc}\ \lfloor n_1\rfloor\ n_2}$$

$$\frac{\langle e_1,\sigma\rangle \Downarrow n_1 \quad \langle e_2,\sigma\rangle \Downarrow n_2 \quad \langle e_3,\sigma\rangle \Downarrow n_3}{\langle \texttt{HiperE}\ e_1\ e_2\ e_3,\sigma\rangle \Downarrow \texttt{Hiper}\ \lfloor n_1\rfloor\ \lfloor n_2\rfloor\ \lfloor n_3\rfloor}$$

$$\frac{\langle e_1,\sigma\rangle \Downarrow xs \quad \langle e_2,\sigma\rangle \Downarrow ps}{\langle \texttt{CustomE}\ e_1\ e_2,\sigma\rangle \Downarrow \texttt{Custom}\ \lfloor xs\rfloor\ ps}$$

8.2.2 Distribuciones Continuas

$$\frac{\langle e_1,\sigma\rangle \Downarrow n_1 \quad \langle e_2,\sigma\rangle \Downarrow n_2}{\langle \texttt{NormE}\ e_1\ e_2,\sigma\rangle \Downarrow \texttt{Norm}\ n_1\ n_2}$$

$$\frac{\langle e_1,\sigma\rangle \Downarrow n_1 \quad \langle e_2,\sigma\rangle \Downarrow n_2}{\langle \texttt{UnifE}\ e_1\ e_2,\sigma\rangle \Downarrow \texttt{Unif}\ n_1\ n_2} \qquad \frac{\langle e,\sigma\rangle \Downarrow n}{\langle \texttt{ExpoE}\ e,\sigma\rangle \Downarrow \texttt{Expo}\ n}$$

8.3 Operaciones Numéricas

$$\frac{\langle e_1,\sigma\rangle \Downarrow \texttt{VNum}\ n_1 \quad \langle e_2,\sigma\rangle \Downarrow \texttt{VNum}\ n_2}{\langle \texttt{OpNum}\ \oplus\ e_1\ e_2,\ \sigma\rangle \Downarrow \texttt{VNum}\ (n_1 \oplus n_2)} \quad \oplus \in \{+,-,\times\}$$

$$\frac{\langle e_1,\sigma\rangle \Downarrow \texttt{VNum}\ n_1 \quad \langle e_2,\sigma\rangle \Downarrow \texttt{VNum}\ n_2 \quad n_2 \neq 0}{\langle \texttt{OpNum}\ /\ e_1\ e_2,\ \sigma\rangle \Downarrow \texttt{VNum}\ (n_1/n_2)}$$

$$\frac{\langle e,\sigma\rangle \Downarrow \texttt{VNum}\ n}{\langle \texttt{UMinus}\ e,\ \sigma\rangle \Downarrow \texttt{VNum}\ (-n)}$$

8.4 Acceso a Vectores

$$\frac{\langle v,\sigma\rangle \Downarrow \texttt{VVec}\ xs \quad \langle e,\sigma\rangle \Downarrow \texttt{VNum}\ n \quad 0 \leq \lfloor n\rfloor < |xs|}{\langle \texttt{Access}\ v\ e,\ \sigma\rangle \Downarrow \texttt{VNum}\ (xs[\lfloor n\rfloor])}$$

8.5 Estadísticas

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X}{\langle \texttt{Mean}\ e,\ \sigma\rangle \Downarrow \texttt{VNum}\ (E[X])} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X}{\langle \texttt{Variance}\ e,\ \sigma\rangle \Downarrow \texttt{VNum}\ (\mathrm{Var}(X))}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X}{\langle \texttt{StdDev}\ e,\ \sigma\rangle \Downarrow \texttt{VNum}\ (\sigma_X)} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X}{\langle \texttt{Mode}\ e,\ \sigma\rangle \Downarrow \texttt{VVec}\ (\mathrm{mode}(X))}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ (\texttt{Cont}\ X) \quad \langle x, \sigma\rangle \Downarrow \texttt{VNum}\ v}{\langle \texttt{PDF}\ e\ x,\ \sigma\rangle \Downarrow \texttt{VNum}\ (f_X(v))}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ (\texttt{Disc}\ X)}{\langle \texttt{MaxP}\ e, \sigma\rangle \Downarrow \texttt{VNum}\ (\max_k P(X = k))} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ (\texttt{Cont}\ X)}{\langle \texttt{MaxPDF}\ e, \sigma\rangle \Downarrow \texttt{VNum}\ (\max_x f_X(x))}$$

8.6 Probabilidades

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X \quad \langle k, \sigma\rangle \Downarrow \texttt{VNum}\ n}{\langle \texttt{Prob}\ e\ \texttt{op}\ k,\ \sigma\rangle \Downarrow \texttt{VNum}\ (P(X\ \texttt{op}\ n))}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X \quad \langle k_1, \sigma\rangle \Downarrow \texttt{VNum}\ n_1 \quad \langle k_2, \sigma\rangle \Downarrow \texttt{VNum}\ n_2}{\langle \texttt{ProbBetween}\ e\ \texttt{op}_1\ k_1\ \texttt{op}_2\ k_2,\ \sigma\rangle \Downarrow \texttt{VNum}\ (P(n_1\ \texttt{op}_1\ X\ \texttt{op}_2\ n_2))}$$

8.7 Cadenas de Markov

$$\frac{\forall\, x \in p\ .\ x \in \mathrm{dom}(\sigma_{nodes}) \quad ns = [\sigma_{nodes}(x) \mid x \in p]}{\langle \texttt{MarkovE}\ p, \sigma\rangle \Downarrow \texttt{VMark}\ (\texttt{Mk}\ p\ (\texttt{makeMatrix}\ p\ ns))}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad \langle n, \sigma\rangle \Downarrow \texttt{VNum}\ \lfloor k \rfloor \quad i, j \in \mathrm{states}(M) \quad k \geq 1}{\langle \texttt{ProbStep}\ e\ i\ j\ n,\ \sigma\rangle \Downarrow \texttt{VNum}\ ((P^k)_{ij})}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad \langle c, \sigma\rangle \Downarrow \texttt{VPath}\ p \quad p_i \in \mathrm{states}(M)}{\langle \texttt{ProbPath}\ e\ c,\ \sigma\rangle \Downarrow \texttt{VNum}\ \left(\prod_{t=1}^{|p|-1} P_{p_{t-1}, p_t}\right)}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad i, j \in \mathrm{states}(M)}{\langle \texttt{ProbHit}\ e\ i\ j,\ \sigma\rangle \Downarrow \texttt{VNum}\ (h_{ij})} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M}{\langle \texttt{Stationary}\ e,\ \sigma\rangle \Downarrow \texttt{VVec}\ \pi}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad \langle n, \sigma\rangle \Downarrow \texttt{VNum}\ k}{\langle \texttt{NextDist}\ e\ n,\ \sigma\rangle \Downarrow \texttt{VMark}\ (M' \text{ con matriz } P^k)}$$

8.8 Simulaciones

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad \langle s, \sigma\rangle \Downarrow \texttt{VNum}\lfloor k \rfloor \quad m \in \mathrm{states}(M) \quad p \sim \mathrm{simulate}(M, m, k)}{\langle \texttt{SimulFromName}\ e\ m\ s, \sigma\rangle \Downarrow \texttt{VPath}\ p}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMark}\ M \quad \langle v, \sigma\rangle \Downarrow \texttt{VVec}\ \pi_0 \quad \langle s, \sigma\rangle \Downarrow \texttt{VNum}\ \lfloor k \rfloor \quad p \sim \mathrm{simulate}(M, \pi_0, k)}{\langle \texttt{SimulFromVec}\ e\ v\ s, \sigma\rangle \Downarrow \texttt{VPath}\ p}$$

8.9 Comandos

$$\frac{\langle e, \sigma\rangle \Downarrow v}{\langle \texttt{Let}\ x\ e,\ \sigma\rangle \Downarrow \sigma_{decls}[x \mapsto v]} \qquad \frac{\langle e, \sigma\rangle \Downarrow v}{\langle \texttt{Print}\ e,\ \sigma\rangle \Downarrow \mathbf{output}(v)}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ (\texttt{Disc}\ X)}{\langle \texttt{Table}\ e,\ \sigma\rangle \Downarrow \mathbf{makeTable}(X)} \qquad \frac{\langle e_1, \sigma\rangle \Downarrow \texttt{VRand}\ (\texttt{Disc}\ X) \quad \langle e_2, \sigma\rangle \Downarrow \texttt{VNum}\lfloor n\rfloor \quad \langle e_3, \sigma\rangle \Downarrow \texttt{VNum}\lfloor m\rfloor}{\langle \texttt{Table}\ e_1, e_2, e_3\ \sigma\rangle \Downarrow \mathbf{makeTable}(X, n, m)}$$

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{VMkar}\ X}{\langle \texttt{Table}\ e\ \sigma\rangle \Downarrow \mathbf{plot}(X)} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{VRand}\ X}{\langle \texttt{Table}\ e\ \sigma\rangle \Downarrow \mathbf{plot}(X)}$$

$$\frac{\langle n, \sigma\rangle \Downarrow \texttt{NodeVal}\ nd}{\langle \texttt{LetN}\ x\ n,\ \sigma\rangle \Downarrow \sigma_{nodes}[x \mapsto nd]}$$

9 Validador

Antes de evaluar cualquier expresión, el intérprete verifica que los valores involucrados sean matemáticamente coherentes. El validador se invoca *durante* del evaluador.

9.1 Valores Numéricos

Cuando una operación requiere un entero, se verifica que el valor proporcionado no tenga parte decimal y esté dentro del rango representable. Si no se cumple, se reporta *IntValueExpected*.

9.2 Variables Aleatorias Discretas

9.2.1 Binomial $\mathbf{Bin}(n, p)$

- $p \in [0, 1]$
- $n \geq 0$

9.2.2 Poisson $\mathbf{Poi}(\lambda)$

- $\lambda \geq 0$

9.2.3 Geométrica $\mathbf{Geo}(p)$

- $p \in (0, 1]$

9.2.4 Pascal $\mathbf{BN}(r, p)$

- $p \in (0, 1]$
- $r \geq 1$

9.2.5 Hipergeométrica $\mathbf{HG}(m, r, n)$

- $m > 0, \quad r > 0, \quad n \geq 0$
- $r \leq m$ (no puede haber más marcados que el total)
- $n \leq m$ (no puede extraerse más de lo que hay)

9.2.6 Distribución Personalizada [xs ps]

- El vector de probabilidades no puede estar vacío.
- Los valores en **xs** deben ser todos distintos y en igual cantidad que las probabilidades en **ps**.
- Cada $p_i \in [0, 1]$.
- $\sum_i p_i = 1$ (con tolerancia 10^{-9}).

9.3 Variables Aleatorias Continuas

9.3.1 Normal $\mathbf{N}(\mu, \sigma)$

- $\sigma > 0$

9.3.2 Uniforme $\mathbf{Unif}(a, b)$

- $a \leq b$

9.3.3 Exponencial $\mathbf{Exp}(\lambda)$

- $\lambda > 0$

9.4 Probabilidades

Al calcular $P(X \ \texttt{op} \ k)$ o $P(k_1 \ \texttt{op}_1 \ X \ \texttt{op}_2 \ k_2)$ se verifica que los valores consultados pertenezcan al soporte de la distribución.

Distribución	**Restricción sobre k**
$\text{Bin}(n, p)$	$0 \leq k \leq n$
$\text{Poi}(\lambda)$	$k \geq 0$
$\text{Geo}(p)$	$k \geq 1$
$\text{BN}(r, p)$	$k \geq r$
$\text{HG}(m, r, n)$	$\max(0,\, n - (m - r)) \leq k \leq \min(n, r)$
Custom	k debe pertenecer al vector de valores

Table 4: Restricciones de soporte para distribuciones discretas.

Para probabilidades con dos operadores se verifica adicionalmente que el intervalo resultante no sea vacío. Para distribuciones continuas solo se verifica esta condición.

9.5 Cadenas de Markov

9.5.1 Definición de la cadena

- Todo nombre referenciado en un nodo debe existir en la cadena.
- Cada probabilidad de transición debe estar en $[0, 1]$.
- La suma de probabilidades de salida de un nodo no debe superar 1.
- No puede haber nombres repetidos dentro del mismo nodo.

9.5.2 Vector de distribución inicial

- Su longitud debe coincidir con el número de estados.
- Cada componente en $[0, 1]$ y la suma igual a 1 (tolerancia 10^{-9}).

9.5.3 Nombres de estado

Todo nombre de estado referenciado debe pertenecer a la cadena.

9.5.4 Caminos

Un camino debe tener al menos dos estados, y cada uno debe pertenecer a la cadena.

9.5.5 Número de pasos

El número de pasos n en consultas $F^n(X, i, j)$ o en simulaciones debe ser estrictamente positivo ($n \geq 1$).

10 Pretty Printer

El sistema incorpora un módulo de pretty printing que permite representar expresiones, valores y comandos de forma legible y coloreada en la terminal. Está implementado sobre la biblioteca `prettyprinter` utilizando el tipo `Doc AnsiStyle`, que permite anotar fragmentos del documento con estilos ANSI (colores y negrita).

El pretty printer es utilizado por el modo `-verbose` para mostrar cada comando evaluado, y por el comando `print` para mostrar el valor resultante de una expresión.

11 Visualización y Representación de Datos

11.1 Generación de Tablas

El comando `table` genera una tabla renderizada en terminal con la distribución de probabilidad de una variable aleatoria discreta. El rango mostrado se centra automáticamente en la **moda** de la distribución para maximizar la información relevante. Con `table(X, lo, hi)` se puede especificar un rango explícito.

11.2 Generación de Gráficos

El comando `plot` tiene dos usos:

- **Funciones de probabilidad:** Genera archivos `.png` con la PMF (discreta) o PDF (continua). Utiliza la librería `EasyPlot` sobre `gnuplot`, que debe estar instalado en el sistema. El gráfico se centra en la moda.
- **Cadenas de Markov:** Genera un grafo dirigido donde cada nodo representa un estado y las aristas las probabilidades de transición. Utiliza la interfaz de `Graphviz`, que también debe estar instalado.

12 Arquitectura de la Mónada: `MonadProb`

Para gestionar el intérprete y sus efectos se define la clase monádica `MonadProb` y su implementación concreta `ProbM`:

```
type ProbM = ReaderT Conf (StateT Env (ExceptT Error IO))
```

Cada capa cumple una función específica:

- `ReaderT Conf`**:** Proporciona un contexto de configuración de solo lectura (`Conf`), con acceso a parámetros como la precisión decimal y el modo *verbose*.
- `StateT Env`**:** Gestiona el estado mutable del sistema: las declaraciones de variables (`decls`) y los nodos de las cadenas de Markov (`nodes`).
- `ExceptT Error`**:** Manejo estructurado de errores con tres categorías: `ParseErr`, `TypeErr` y `ExecErr`.
- `IO`**:** Necesaria para la lectura de archivos, la generación de números aleatorios y la salida por terminal.

La clase de tipos `MonadProb` abstrae estas capacidades:

```
class ( MonadIO m
      , MonadState Env m
      , MonadError Error m
      , MonadReader Conf m
      ) => MonadProb m
```

Esto permite que el typechecker, el validador y el evaluador sean polimórficos en la mónada, facilitando el testing y la extensión del intérprete.

13 Estructura del Proyecto

13.1 Organización de Archivos

El proyecto sigue la estructura estándar de un proyecto Cabal, con el código fuente dividido en módulos según su responsabilidad.

```
tp-final-alp/
├─ app/
│  └─ Main.hs
├─ src/
│  ├─ AST.hs
│  ├─ CBin.hs
│  ├─ Common.hs
│  ├─ Distribution.hs
│  ├─ Error.hs
│  ├─ Eval.hs
│  ├─ Global.hs
│  ├─ Markov.hs
│  ├─ Matrix.hs
│  ├─ Monad.hs
│  ├─ Parse.hs
│  ├─ Plot.hs
│  ├─ PPrint.hs
│  ├─ Simulation.hs
│  ├─ Table.hs
│  ├─ TypeChecker.hs
│  └─ Validator.hs
└─ tp-final-alp.cabal
```

14 Ejecución

14.1 Paso de Archivos y Configuración

Para ejecutar un archivo de script se pasa la ruta como argumento. El programa acepta las siguientes banderas de configuración:

- `-verbose | -v`: Activa el modo detallado, mostrando cada comando evaluado con su resultado.
- `-decimals <n> | -d <n>`: Define la precisión decimal de los resultados numéricos. El valor por defecto es **6**.

Ejemplo de ejecución:

```
cabal run tp-final-alp -- -verbose -decimals 4 mi_script.prob
```

14.2 Ejemplos

Se provee una carpeta `Examples/` con scripts de ejemplo que cubren los distintos casos de uso del lenguaje.

14.3 Modo Interactivo (REPL)

Si el programa se inicia sin archivos, entra automáticamente en modo **REPL**. El estado persiste entre líneas: las variables declaradas en una instrucción son accesibles en las siguientes. El historial de comandos se guarda automáticamente en `.prob_history`.

Los comandos disponibles dentro del REPL son:

Comando	Acción
`:q`, `:quit`	Finaliza la sesión.
`:help`	Muestra la lista de comandos.
`:env`	Muestra las variables declaradas en el estado actual.
`:load <file>`	Carga y ejecuta un archivo externo.
`<command>`	Evalúa un comando del lenguaje.

Table 5: Comandos del modo interactivo.

14.4 Ejemplo de Sesión

```
prob> X ~ Bin(10, 0.3)
prob> print(E(X))
3.0
prob> print(V(X))
2.1
prob> print(P(X <= 3))
0.649612
prob> table(X)
+----------+
|  Bin     |
+----------+
| 0 | 0.02 |
| 1 | 0.12 |
| 2 | 0.23 |
| 3 | 0.27 |
...
prob> :q
Bye!
```

15 Bibliografía y Recursos

1. **Hackage — The Haskell Package Repository.** Repositorio oficial de paquetes de Haskell: `https://hackage.haskell.org/`
2. **Cátedra de Análisis de Lenguajes de Programación.** Slides y trabajos prácticos de la materia.
3. **Cátedra de Probabilidad y Estadística.** Slides y notas de la materia.
4. **Wikipedia.** consultada como referencia rápida para propiedades matemáticas de distribuciones de probabilidad: `https://www.wikipedia.org/`
5. **Compilador FD4 — Cátedra de Compiladores.** Implementación de referencia utilizada como base estructural del proyecto.
6. **Asistentes de Inteligencia Artificial: Claude, Gemini y ChatGPT.** Herramientas de IA utilizadas como apoyo durante el desarrollo del proyecto en las siguientes tareas: búsqueda y comparación de paquetes Haskell para generación de gráficas; consulta y explicación de conceptos estadísticos; generación y corrección de documentación; y revisión de código.

16 Mejoras futuras

En esta sección incluyo algunas ideas que consideré implementar, pero que finalmente decidí dejar de lado para no complejizar el código.

1. **Optimización de consultas:** Para las variables discretas, actualmente usamos funciones como el factorial o el coeficiente binomial con un costo $O(n)$ para obtener la probabilidad en un punto. El problema es que, al consultar un rango k, el costo se dispara porque llamamos a la función k veces. Una alternativa sería aprovechar que ya usamos funciones recursivas y, en lugar de recalcular cada factorial por separado, utilizar algo como `scanl (*) 1 [1..n]` para obtener todos los resultados intermedios de una sola vez. También se podría elegir distintas estrategias dependiendo de la ditribución o del rango.
2. **Modo verbose detallado:** Me gustaría que al activar la bandera `verbose` no solo se muestre el comando y su resultado, sino todo el rastro de la evaluación. Pensé en usar la mónada `Writer`, pero no basta con registrar cada paso, ya que se imprimiría cada subárbol de la expresión y el resultado sería muy confuso. Implementar un "verbose" realmente útil requeriría un módulo de evaluación distinto, capaz de mantener el contexto y reconstruir la ejecución de forma legible.